ECE 5730 Lab 3 Report

1D-Helicopter



Amber Li | Ivan Huang | Zarif Karim

10.27.2026

Introduction



Figure 1: Lab Overview

Mathematical Background

PID

Physics

Fixed Point Arithmetic

Although the Pico 2's CPU contains a Floating Point Unit (math block that works on decimals), it is actually still faster to use fixed point arithmetic. Running through the ALU required less steps to compute on purely binary numbers.

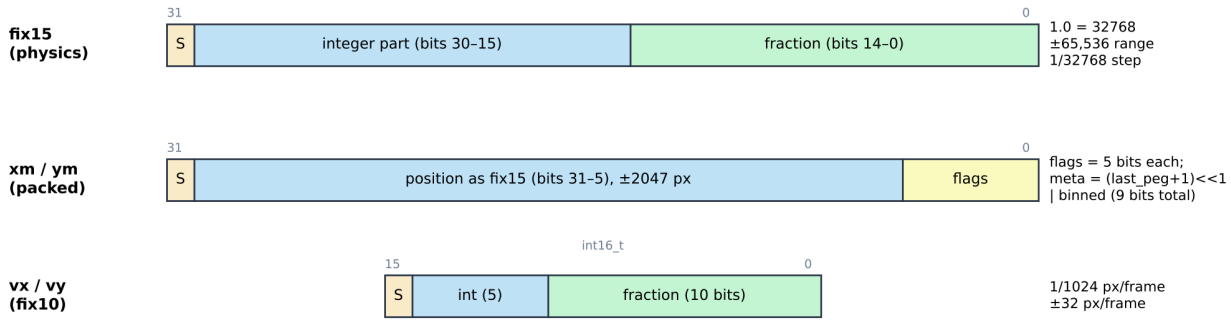


Figure 3: Fixed-point formats. Top: fix15 (signed 17.15) used for every physics calculation. Middle and bottom: the packed position and velocity formats inside our 12-byte ball struct.

Physics calculations were done using what we call “fix15.” In this case, it still looks as if it is a signed 32 integer, but it's actually split into 3 sections: MSB still signifies positive/negative, but now the 15 in fix means at the 15th bit, bits 14 → 0 represents bits of a fractional number. The remaining 16 bits 30 → 15 represent a 16 bit binary number.

Design and Testing Methods

MPU-6050

Timer Interrupt

VGA Driver (Pre optimization)

In order to display graphics, we developed both software and hardware for this purpose. We not only built upon a VGA driver provided by Hunter Adams, but also we soldered a custom PCB that streamlined required VGA connections. Below is a snapshot of the PCB used:

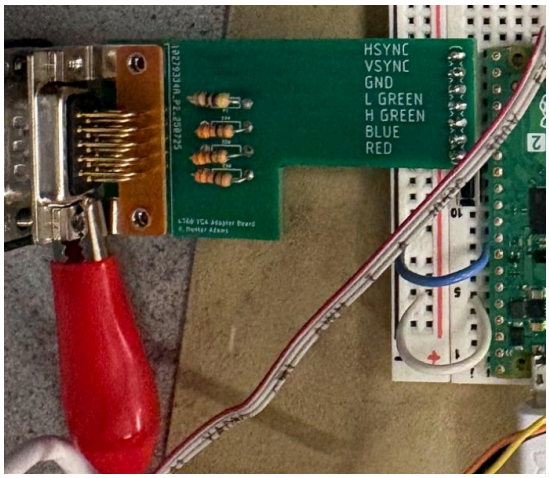


Figure 7: VGA PCB

Afterwards we then built upon a given VGA demo. We were given a double-buffered version of the course VGA driver. Three PIO state machines generate hsync, vsync, and the 4-bit color output, and a DMA chain feeds them pixel data. At every vsync time, the DMA chain tells the buffer we just finished drawing, points at the other buffer, and sets a flag. This gives both cores about 16,667 µs (~16 ms) to draw each frame. If they run late, the half-drawn buffer gets shown anyway so the frame is torn rather than slowed down.

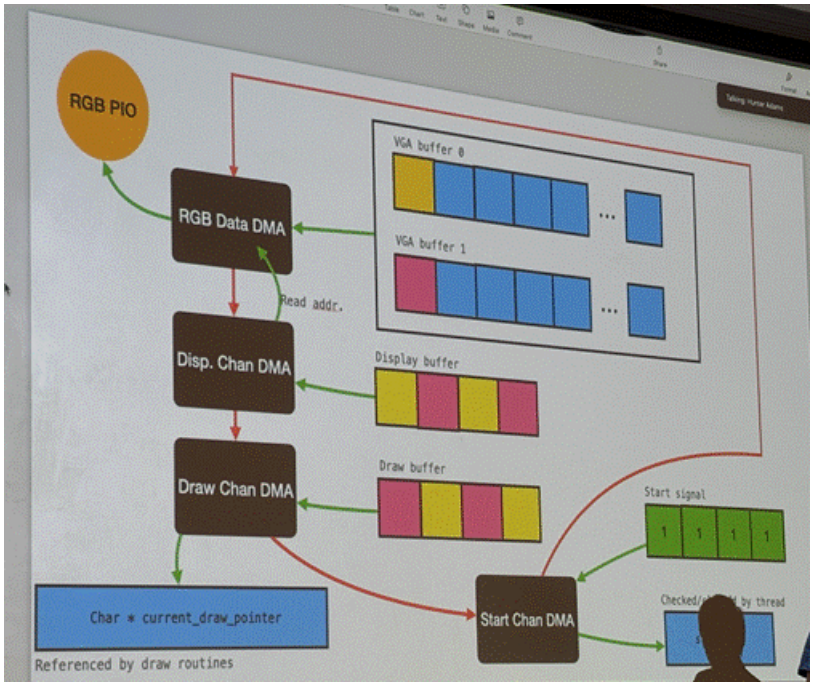


Figure 8: VGA Controller Block Diagram

In the initial VGA demo folder was a working double-buffered, 16 bit color driver where two balls are bouncing inside a rectangle area.

Frame buffer layout works like thisL one frame is 640 by 480 by 4 bits. Stores row by row. That ends up being 320 bytes per row. Two pixels share a byte, one pixel takes the lower half and other takes the higher half. This is important late in our optimizations, such as our ball sprite layout, figuring out how to write whole bytes to draw two pixels simultaneously. Another reason this is important is because we have two buggers, about 300 of the 520 kB of SRAM is used, causing RAM to become another limiting factor of maximum balls.

PID Implementation

How you selected the three PID gains.

P - look for edge of oscillation between stability (oscillations decrease to stable arm) and instability (oscillations increase to massive swinging behavior)
D - should stop the oscillations. When pressing down below target angle, should spring back up and overshoot once before quickly settling to a stable position.
I - should raise the arm to remove the steady state error and allow for reaching the target angle. Increase to reach the target angle quicker (what if its too high?)

What are the maximum/minimum angles that produce stable behavior for your PID gains?
Max angle = 50 (varies with 5V power supply used)
Min angle = (motor off?) -90

What are the settling times at different angles?

Target Angle (°)	Settling Time (s)
-90 to 0	2.74
0 to 30	2.38
30 to -30	3.56
-30 to 0	0.98

Table _: Settling times between angles in the sequence.

Final Wiring

Below is the table of connections:

Signal	**RP2350 pin**	**Notes**
VGA Hsync / Vsync	GPIO 16 / 17	PIO state machines
VGA Green lo / Green hi / Blue / Red	GPIO 18 / 19 / 20 / 21	470 Ω / 330 Ω / 330 Ω / 330 Ω
Encoder A / B	GPIO 12 / 11	internal pull-ups, 0.1 µF to GND each, edge IRQ
Encoder push button	GPIO 10	active high, internal pull-down, polled
MCP4822: CS / SCK / MOSI	GPIO 13 / 14 / 15	spi1, fed by DMA
On-board LED	GPIO 25	turns on when a frame misses 60 fps

Table 2: RP2350 Wiring Connections

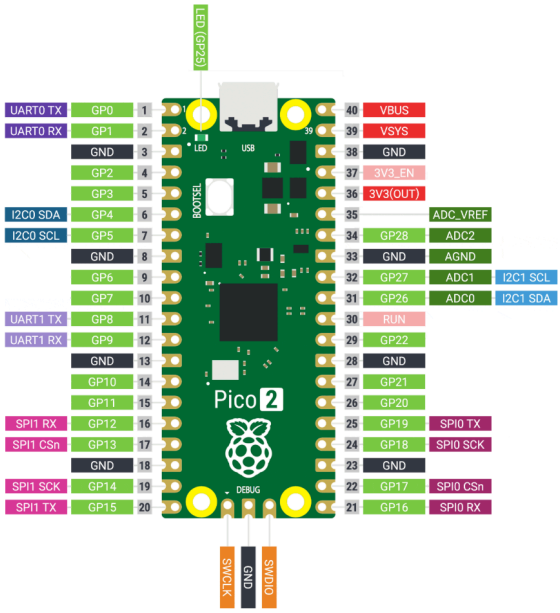


Figure 11: RP2350 Pinout

Think about it: Every new row adds an extra random step to the sum. The central limit theorem is **Application Level**

Figure 21: System Level Block Diagram

A schematic of the circuit you built

Here is a picture of our circuit breadboarded:

Figure 22: Breadboard Hardware Setup

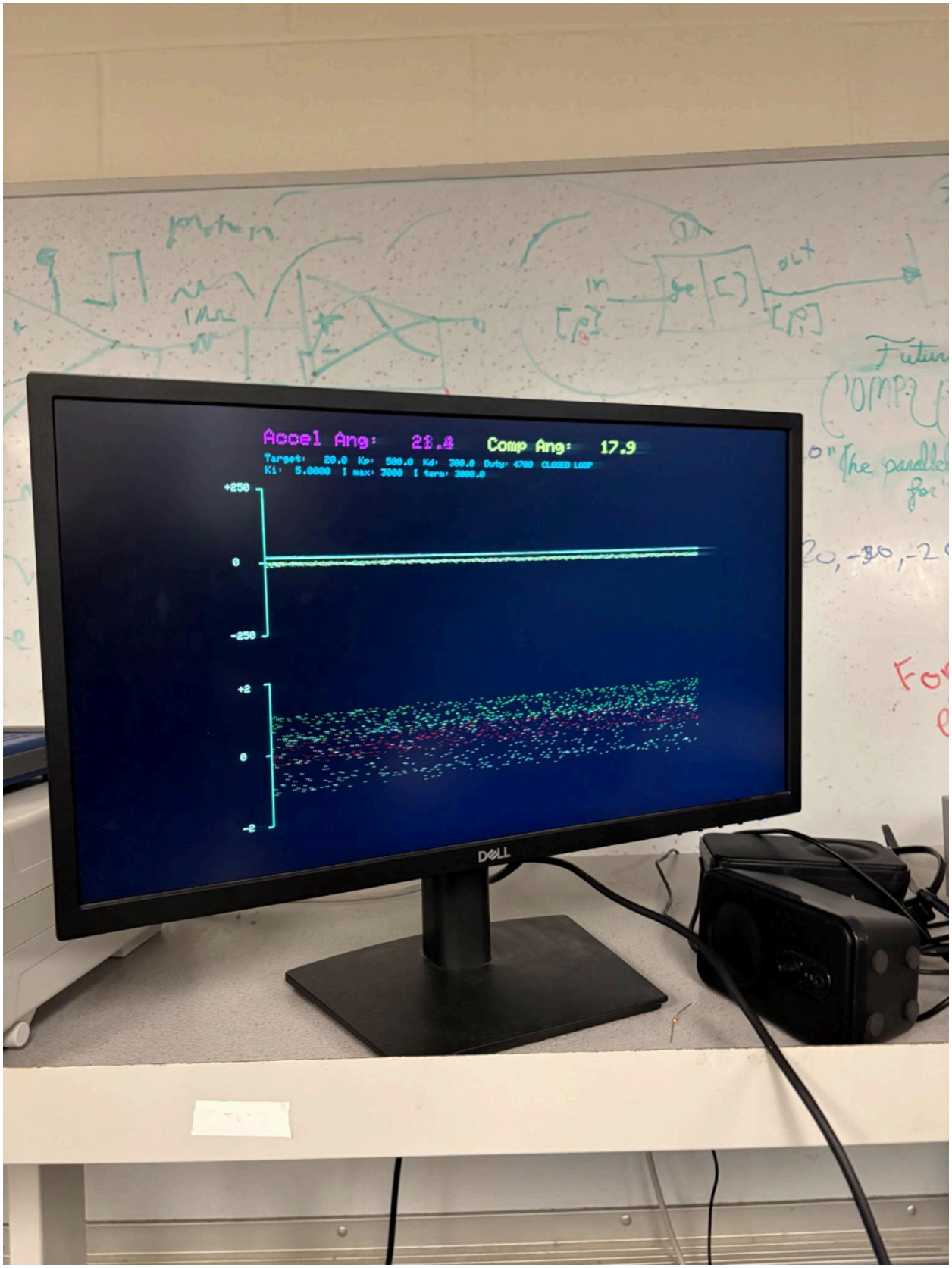
VGA screen dumps of typical two-trace: (1) motor-control and (2) actual angle. Include the demo sequence of four angles, as shown below

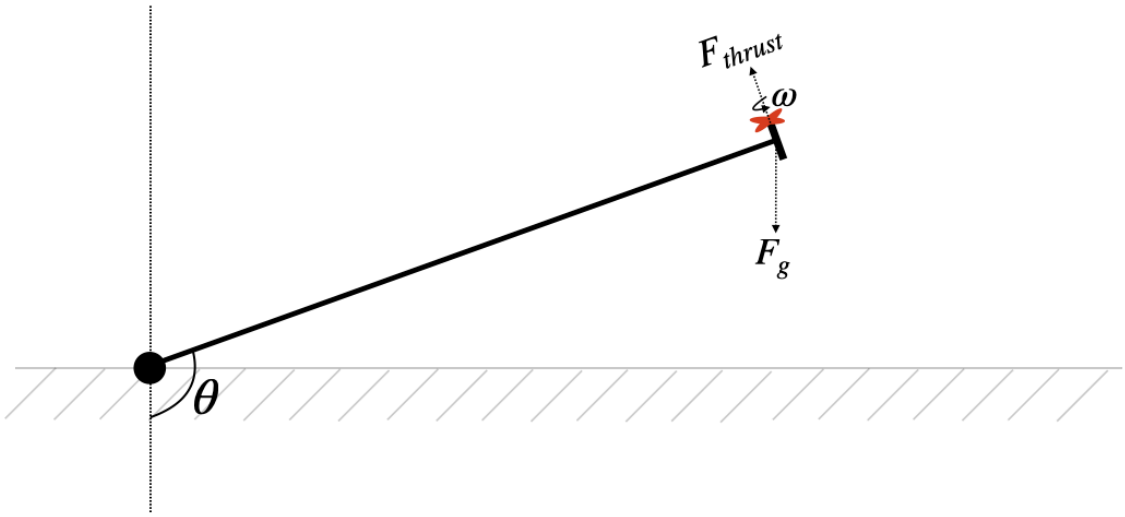
Conclusion:

Work Distribution:

Amber, Ivan, and Zarif worked together on all parts of the lab. We frequently met outside of our lab sections, and most of our work was done in person in the lab. Outside of the lab, we communicated frequently to ensure that we were all aware of the progress that had been made and what the immediate next steps were. The contents of this report were written by us.

Pictures to use




F thrust
ω
F g
θ